

Assignment 6 (Week 6): Singly Linked Lists — Nodes, Links, Pointer Surgery

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 26, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 6’s Notes chapter alone. Notation matches the lecture exactly: `struct node { int data; struct node *next; };`, `head` is the first-node pointer (NULL when empty), `p->next` is the successor link, NULL terminates the chain, positions are 0-based, and the reverse loop uses `prev`, `curr`, `next`. For every bound, name the operation, give the bound, and state the case (Week 2 shape).

6.1 Conceptual

Q1. A class register holds 2000 student IDs in an array. During admission week the office inserts 400 new IDs at the front, one at a time; afterwards it looks up IDs by roll number only occasionally (Sedgewick Sec. 1.3, p. 142; Karumanchi Ch. 3, pp.74–162 PDF) [2, 1].

- (a) In one or two sentences, explain the **slot tax** on this workload: what does one front-insertion into a full array of n cost worst case, and why?
- (b) In one or two sentences, explain what the singly linked list changes: which single pointer rewrite replaces the shift, what does front-insertion cost worst case, and what is lost in return (name the operation whose bound gets worse and both bounds)?
- (c) In one or two sentences, state the workload rule and apply it: which structure fits the admission-week workload, and name one opposite workload on the same register for which you would keep the array.

Why this matters: the register is big enough that the wrong choice bills 400 full shifts — this question forces you to price the workload, not the structure, before choosing.

Q2. Two orderings, one theme: never destroy the only path before saving it (Karumanchi Ch. 3, pp.74–162 PDF) [1].

- (a) Front-insertion runs `newNode->next = head then head = newNode`. In one or two sentences, explain precisely what is orphaned if the two assignments are swapped, and why no copy of the lost address survives anywhere.

- (b) Iterative reverse runs `next = curr->next` *before* `curr->next = prev`. In one or two sentences, explain what is lost if the flip comes first, using the chain `head → 40 → 50 → 60 → NULL` with `curr` on 40 as the concrete instance.
- (c) In one or two sentences each, name the Week 1 bug each violation becomes if the program keeps running: the swapped front-insert (heap still billed, nobody pointing) and using `temp` after `free(temp)` in delete.

Why this matters: both classic list bugs are the same single mistake wearing different clothing — this question makes you state the mistake once so you recognise it twice.

6.2 Numerical

Q3. Start from `head → 40 → 50 → 60 → NULL`. Rules: front-insert is the deck's two-write sequence (`newNode->next = head` first); position insert walks `p` from `head` to the predecessor then splices in the same order; delete bypasses then frees exactly once.

- (a) Front-insert 35. Give the heap chain after the step and the two assignments in order.
- (b) On the result, insert 55 at position 2 (0-based). Which node does `p` stop on, what are the two splice assignments in order, and what is the final chain?
- (c) On the result, delete the first occurrence of 50. Write the bypass assignment, name the freed node, give the final chain, and state the worst-case bound for each of (a)–(c) with operation, bound, and case.

Why this matters: insert, splice, and bypass are the same two-write surgery in three positions — this question puts your hand through all three on one chain the lecture never traced.

Q4. Start from `head → 7 → 14 → 21 → 28 → NULL`. Search and display are the deck's two walks (`p = head; while (p != NULL ...)`); reverse is the three-pointer loop (`prev = NULL; curr = head; save next, flip, advance both walkers; head = prev`).

- (a) How many nodes does `search(21)` visit, and how many does `search(99)` visit? Name the case behind each count and give the worst/best bounds for search.
- (b) What does `display()` print, does `head` move, and what is `display`'s bound?
- (c) Reverse the list with the three-pointer loop. Give `prev/curr/next` after *every* iteration as a four-row table, the final `head`, and the time and auxiliary-space bounds with cases.

Why this matters: counting visits is pricing, and the reverse table is the exam's favourite trace — this question makes you produce both on a chain the notes never worked.

6.3 Design

Q5. Design sorted insertion into an ascending singly linked list (a *new* scenario: the lecture inserts at a given position, never by key). Assume distinct keys, no tail pointer (Karumanchi Ch. 3, pp.74–162 PDF) [1].

- (a) Write pseudocode `sortedInsert(head, x)` that walks with `p` (keeping its predecessor), then splices with the deck's two-write order. Handle all three landings explicitly: empty list / insert at front (`x` smallest), middle splice, insert at end (`x` largest). Include the `malloc-for-NULL` check and the `p != NULL` walk guard.

- (b) Trace `sortedInsert` with $x = 29$ on `head` $\rightarrow 11 \rightarrow 22 \rightarrow 33 \rightarrow 44 \rightarrow \text{NULL}$: which node does the walk stop on, what are the two splice assignments, and what is the final chain?
- (c) State the worst-case bound with operation, bound, and case, name which step dominates (walk vs. splice), and give the final chain for the edge inputs $x = 5$ and $x = 50$ on the same starting list.

Why this matters: the calculator’s symbol table wants IDs kept in order without re-sorting the world — this question extends the walk-then-splice pattern to the key-ordered case the lecture left open.

Q6. Design tail deletion plus in-place reversal, with guards (both *new* compositions: the lecture deletes by value and reverses a fixed 3-node chain; here the value is “last” and the chain is fresh). No tail pointer. List `head` $\rightarrow 6 \rightarrow 12 \rightarrow 18 \rightarrow 24 \rightarrow \text{NULL}$ (Karumanchi Ch. 3, pp.74–162 PDF) [1].

- (a) Write pseudocode `deleteLast(head)` that returns the new `head`: handle the empty list (return `NULL`), the single-node list (free it, return `NULL`), and the general case (walk `p` to the second-last node, set `temp = p->next`, bypass with `p->next = temp->next`, free `temp`). State the worst-case bound with case.
- (b) Trace `deleteLast` on $6 \rightarrow 12 \rightarrow 18 \rightarrow 24$: which node does `p` stop on, what is the bypass assignment, which node is freed, and what is the surviving chain?
- (c) Reverse the surviving chain in place with the three-pointer loop (no copying into a second list). Give the final `head`, the time and auxiliary-space bounds, and in one or two sentences explain why copying into a second list would double the memory for a mere rearrangement (the playlist argument from the deck).

Why this matters: “delete the last job, then play newest first” is the whole week as one shipping task — guards first, surgery second, pricing throughout.

References

- [1] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [2] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.